

Service Lifecycle Operations

CYBOL Operation \ Channel	file directory	serial port	terminal	named pipeline	client socket	server socket	display (xcb)	display (win32)	mobile app
maintain/startup (server)	-	-	-	-	-	+	+	+	
[memorise/allocate (server entry)]	-	-	-	-	-	+	+	+	
configure/initialise (service)	-	-	-	-	-	+	+	+	
attend/enable (accept thread)	-	-	-	-	-	+ (handler)	+ (global event loop)	-	
dispatch/open (client)	+	+	+	+	+	+ (in enable handler)	+	+	
[memorise/allocate (server entry)]	0	0	0	0	0	-	-	-	0
[memorise/allocate (client entry)]	+	+	+	+	+	+	+	+	+
adjust/prepare (device)	+	+	+	+	+	- (configured by server)	+	+	+
feel/sense (message thread)	-	0	0	0	0	+ (handler)	+ (handler, NO thread)	+ (handler, USE thread)	
communicate/receive (data)	+	+	+	+	+	+	+	+	+
stream/read	+	+	+	+	+	+	+	+	+
pack/extract	0	0	0	0	0	0	-	-	-
convert/decode	0	0	0	0	0	0	-	-	-
represent/deserialise	0	0	0	0	0	0	+	+	+
(process data in various ways)									
represent/serialise	0	0	0	0	0	0	+	+	+
convert/encode	0	0	0	0	0	0	-	-	-
pack/compress	0	0	0	0	0	0	-	-	-
stream/write	+	+	+	+	+	+	+	+	+
communicate/send (data)	+	+	+	+	+	+	+	+	+
feel/suspend	-	0	0	0	0	+	+	+	
adjust/postpare (device)	+	+	+	+	+	-	+	+	+
[memorise/deallocate (client entry)]	+	+	+	+	+	+	+	+	+
[memorise/deallocate (server entry)]	-	-	-	-	-	-	-	-	0
dispatch/close (client)	+	+	+	+	+	+	+	+	
attend/disable	-	-	-	-	-	+	+	-	
configure/finalise (service)	-	-	-	-	-	+	+	+	
[memorise/deallocate (server entry)]	-	-	-	-	-	+	+	+	
maintain/shutdown (server)	-	-	-	-	-	+	+	+	
Legend of cybol operation calls:									
+ mandatory									
o optional									
- none									
[] cyboi-internal function									
Colour of cybol calls:									
yellow – standard wrapped operation,	sums up special operations, for convenience								
white – special detailed operation, for	fine-tuning (seldom)								
Display on Windows OS:									
In the x window system (xcb), there is	just one global function delivering events of all windows.								
In win32, there is one thread per window,	with a separate event function.								
Therefore, do NOT call attend/enable,	but only buffer/activate + feel/sense for each new win32 window that was created with dispatch/open.								

Service Lifecycle Operations

General Communication:
For each communication, client as well as server, cyboi allocates a SERVER ENTRY to store the channel base + service port as well as a client list.
For client communication, cyboi allocates a CLIENT ENTRY to store the client id and buffer for asynchronous communication.
This is done implicitly when starting up a server or opening a client.
In parallel to the client list, there are other lists keeping the device id (filename, socket number, window id) and timeout numbers.
It does not matter if a client gets created automatically e.g. by a server socket or manually e.g. as window.
Its identification (id) has to be stored so that cyboi can free resources properly at system exit.
If e.g. for a file, the path and filename are given in cybol, then cyboi looks internally for the correct client id to which the filename is mapped.
That way, cyboi can close ALL resources at system exit even if the developer forgot to call the cybol dispatch/close or shutdown operation.
Display:
open each window and buffer manually
catch events of ALL windows using global function of window system
store window id of window that the event stems from in cybol app
map window id to cyboi-internal client id
store window id (as „device“) in client entry with client id
assign event to correct buffer of the corresponding client id in cyboi
use irq pipe to inform main thread
read sender (client id) from irq pipe in order to assign correct handler
in cybol handler, use client window id (already stored inside cyboi together with mapped client id before), to send back a gui to the window
The feel/sense operation may be called for each window separately, in order to assign a specific handler (corresponds to the controller in MVC pattern).
However, a new thread MUST NOT be started for the feel/sense function, since the global attend/enable catches all events.
This can be achieved by distinguishing channels in the feel/sense cyboi function, to NOT create a thread for display.
Serial Port and Terminal Client:
They may be used as standalone client, so that a call of dispatch/open and stream/write is enough for synchronous (blocking) communication.
However, for asynchronous communication the buffer/activate and feel/sense need to be called as well.
The client entry allocated by buffer/activate has to be stored in a server entry, so that it can be retrieved again later.
Therefore, maintain/startup (server) is called automatically by the client inside cyboi, so that a server entry gets allocated.
The feel/sense takes a handler as cybol property.
Actually, dispatch/open and buffer/activate could call feel/sense automatically in cyboi.
But there are terminals that use direct blocking input and do not need a sensing thread at all.
Other terminals are just for output and do not need any input at all.
Therefore, the buffer/activate and feel/sense operations may be called optionally, but separately, after dispatch/open.
They are separate operations and not combined so that the thread may be suspended temporarily without having to delete and deallocate buffer resources.
Server Socket:
The client socket id that was accepted in attend/enable gets handed over via pipe to the cyboi main thread and there it gets assigned to the cybol property „sender“.
Within the attend/enable handler, the dispatch/open and feel/sense operations may relate to this cybol property „sender“, which is the client socket id.
Asynchronous Communication:
The operations buffer/activate and feel/sense are only necessary, if asynchronous communication is used.
In this case, buffer/activate will provide two internal buffers, choosable by two flags: one for input and another one for output.
With synchronous (direct) communication, they are not needed and communicate/receive or communicate/send receive data directly from cybol.
Example: Communication via terminal with standard blocking input/output.
With asynchronous (indirect) communication, data are written to and/or read from buffer.
The „asynchronous“ flag needs to be handed over to read and write, so that they can decide whether to read directly (sync) or from buffer (async).